FOR COMPLETE BEGINNERS

Catch it and keep going.

Complete Modern Java, episode 18: exceptions, in Java and TypeScript

> Read a stack trace, catch the problem, and let the program carry on.

One product with a negative price stopped the whole program.

try and catch handle the problem, and the program continues after the catch.

In this episode

- An uncaught exception
- Reading a stack trace
- try and catch
- getMessage()
- Both paths
- parseInt and NumberFormatException
- TypeScript catch and unknown

Coming next

- ArrayList: a list that can grow (episode 19)

At the time of recording: Java 25, Node 24 LTS, IntelliJ IDEA 2026.1.3.

01 What each run printed

Checkout.java: new Product("Eggs", -5)	`IllegalArgumentException; nothing after it prints`
SafeCheckout.java: price -5	`Could not add eggs: price cannot be negative, Checkout continues`
SafeCheckout.java: price 30	`Added Eggs, Checkout continues`
Quantity.java: "two", then "2"	`Not a number: two, then Quantity: 2`
store.ts: e.message	`tsc TS18046: e is of type unknown`
parse.ts: parseInt("two")	`NaN (no exception)`

Every row was run, not predicted.

02 Read the trace from the top

The first line is the kind of problem and the message. The top "at" line is where it was thrown; the lines below show how the program got there.

Here: main (line 2) called the constructor, which called setPriceCents (line 20), which threw.

java Checkout.java

```
Exception in thread "main" java.lang.IllegalArgumentException: price cannot be negative
    at Product.setPriceCents(Product.java:20)
    at Product.<init>(Product.java:7)
    at Checkout.main(Checkout.java:2)
```

> Printed exactly this trace with Episode 17's Product.java.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

03 try and catch

At the throw, Java leaves the try block at once, runs the matching catch, then carries on after it, never back at the failed line.

Catching does not make the bad value good: eggs were not added. With a good price, the catch is skipped.

SafeCheckout.java

```
void main() {
    try {
        Product eggs = new Product("Eggs", -5);
        IO.println("Added " + eggs.getName());
    } catch (IllegalArgumentException e) {
        IO.println("Could not add eggs: " + e.getMessage());
    }
    IO.println("Checkout continues");
}
```

> Printed Could not add eggs: price cannot be negative, then Checkout continues; with 30, Added Eggs then Checkout continues.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

04 TypeScript: check what you caught

JavaScript can throw anything, so the caught value is unknown: check it is an Error before reading message. parseInt("two") returns NaN instead of throwing.

store.ts

```
try {
    const eggs = new Product("Eggs", -5);
    console.log("Added " + eggs.name);
} catch (e) {
    if (e instanceof Error) {
        console.log("Could not add eggs: " + e.message);
    }
}
console.log("Checkout continues");
```

> Without the check, tsc reported TS18046; with it, tsc was quiet and node printed both lines.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

Handle a problem

1. Read the type and message first.
2. The top "at" line: where it was thrown.
3. try the risky lines; catch the kind of problem.
4. The program continues after the catch.
5. Challenge: One, parseInt("two"), Two in try; Caught; Three. What prints?

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-11-07 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.